

Course title: Programming in C++

Page 1 of 12 pages

Course number: 02393

Aids allowed: All aids allowed without Internet (and Generative AI)

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 15 tasks

Submission details:

1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one file per exercise**, with **these exact names**:
 - For exercises 1 to 3, **exZZ-library.cpp**, where **ZZ** ranges from 01 to 03 (i.e., one per exercise);
 - For exercise 4, **ex04-library.zip**, which must contain **ex04-library.cpp** and **ex04-library.h**.
3. You can test your solutions by running the local autograder. To do so, please refer to the Local Grader Guide.
4. Additional tests may be run on your submissions after the exam.
5. Feel free to add comments to your code.
6. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises; if you cannot complete one task, make sure that the code for that exercise compiles.

(a) Implement the function:

```
Tile** createMinefield(unsigned int m, unsigned int n)
```

It must return an array of $m \times n$ Tiles, i.e., `Tile**`. It must allocate the required memory, and initialise each tile with both `mine` and `hit` set to `false`.

(b) Implement the function:

```
bool placeMine(Tile **field, int m, int n, int r, int c)
```

where `field` is a field of size $m \times n$, `r` and `c` are a row and column position (starting from 0).

The function tries to place a mine in the field on the tile at position `(r,c)`. The function must firstly check whether:

- less than one fourth of the total tiles in the field (truncating all decimals) *already* contain a mine;
- the position `(r,c)` is within the boundaries of the field;
- the tile at position `(r,c)` is not occupied by another mine;
- none of the tiles in the field has been touched.

If any of these conditions is *not* satisfied, the function must return `false` without altering the field. If all conditions are satisfied, the function must update the field `field` by placing a mine in the tile at position `(r,c)`, and then return `true`.

(c) Implement the function:

```
void displayMinefield(Tile **field, unsigned int m, unsigned int n)
```

The function must display the contents of all tiles to the second player, according to the explanation at the beginning of this exercise: given a field `field` of size $m \times n$, the field must be displayed as shown in **Fig. 2** of page 2.

Hint. To implement this function, you may want to take inspiration from `revealMinefield`. You may also want to create a helper function to count how many tiles confine with a given tile and contain a mine.

This exercise continues on the next page...

(d) Implement the function:

```
bool isGameOver(Tile **field, int m, int n)
```

where `field` is a field of size $m \times n$.

The function determines if the game is over. The function must check whether:

- At least one of the tiles containing a mine has been touched;
- All the tiles *not* containing a mine have been touched.

If any of these conditions is satisfied, the function must return `true`. Otherwise, the function must return `false`.

EXERCISE 2. DTU FOOTBALL TOURNAMENT

Bob is writing a program to manage the DTU Football Tournament. Each DTU department can form a team, and play a match against another team; the winner of a match proceeds to the next match. Bob's plan is to represent the DTU Football Tournament data as a tree, where the leaves are the teams, and each internal node represent a football match between a "left" and "right" team, and its winner.

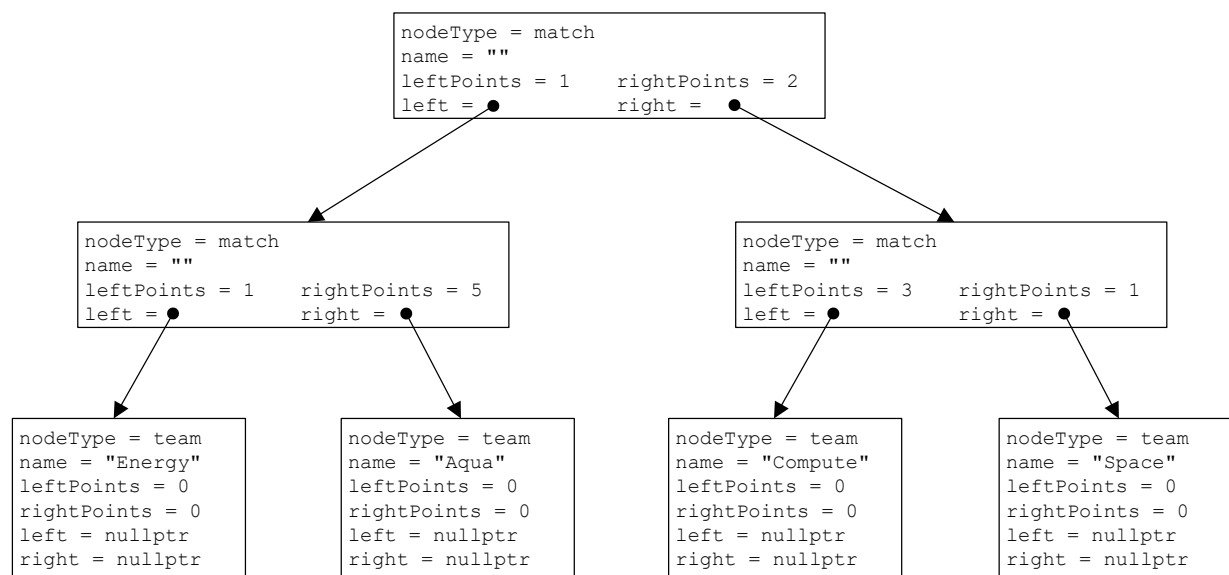
Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. These files are available inside the `handin.zip` archive, under `exercise2footballtournament` folder.

Structure of the code. A tournament tree node is represented with a `struct TournamentNode` having 6 fields:

- `nodeType` can be either `team` or `match`;
- `name` (*only meaningful if `nodeType` is `team`*) is the name of the team;
- `leftPoints` and `rightPoints` (*only meaningful if `nodeType` is `match`*) are the points that each participant of the match scored;
- `left` and `right` (*only meaningful if `nodeType` is `match`*) point to two subtrees.

An empty tournament is represented as a `TournamentNode*` pointer equal to `nullptr`.

Example. A small tournament with 4 teams is represented as follows:



This exercise continues on the next page...

The leaves contain the 4 teams: Energy, Aqua, Compute, and Space. The node with “`nodeType = match`” pointing to Energy and Aqua means: the result of the match was a win for the team on the “right” (i.e. Aqua), since it scored more points than the team on the “left”, which proceeds to the next match. Instead, the match between Compute and Space was won by the team on the “left” (i.e. Compute), since it scored more points than the team on the “right”, which proceeds to the next match. The topmost node is the final match between the winners of the two matches below (i.e. Aqua on the left, and Compute on the right); the result was a win for the team on the “right,” so DTU Compute won the football tournament.

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
string winner(TournamentNode *t)
```

which returns the name of the team that won the tournament `t`. You can assume that no match ended in a draw (i.e., one team always scored more points than the other team).

The function must traverse the match nodes of the tournament tree `t` by descending either on the “left” or “right” (depending on who won the match), until it reaches a team node (which is the winner of `t`).

Special cases. If `t` is `nullptr` or a team, the function must return the string `""`.

Example. Consider the tournament tree on page 5, and assume that `t` points to its topmost node. Then, `winner(t)` must return `"Compute"`.

(b) Implement the function:

```
unsigned int highestScore(TournamentNode *t)
```

which returns the maximum points scored during a match, by a team, in the tournament `t`.

Special case. If `t` is `nullptr`, the function must return 0.

Example. Consider the tournament tree on page 5, and assume that `t` points to its topmost node. Then, `maxPoints(t)` must return 5. Instead, if `t` points to the rightmost match node, then `maxPoints(t)` must return 3.

Hint. You need to explore all match nodes in the tree, keeping track of the maximum points encountered during the exploration.

This exercise continues on the next page...

(c) Implement the function:

```
bool lostAllMatches(TournamentNode *t, string teamName)
```

which returns `true` if the team named `teamName` lost *all* the matches in the tournament `t`. Otherwise (i.e. if the team did not participate in the tournament, or won at least its very first match) the function returns `false`.

Special cases. If `t` is `nullptr` or `t` points to a team node, then the function must return `false`.

Example. Consider the tournament tree on page 5, and assume that `t` points to its topmost node. Then:

- `lostAllMatches(t, "Energy")` must return `true`;
- `lostAllMatches(t, "Aqua")` must return `false`;
- `lostAllMatches(t, "Physics")` must return `false`.

Hint. To find out whether a team called "`X`" has won at least one match in a tournament tree `t`, you can traverse the match nodes of `t`, looking for one where the winner between `left` and `right` directly points to a team node for "`X`". For example, consider the tournament tree on page 5: we can determine that "`Energy`" has lost all matches by traversing the tree until we encounter the match node on the left, whose `left` field (i.e. the loser) directly points to the "`Energy`" team.

EXERCISE 3. UNIVERSITY CAMPUS

Claire is writing a class `Campus` to analyze the effects of renovation work on a university campus. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. These files are available inside the `handin.zip` archive, under `exercise3universitycampus` folder.

Structure of the code. Claire has represented the information about a building of the campus using a `struct Building`, with 3 fields:

- `name`: the name of the building;
- `purpose`: what is the purpose of the building;
- `landOccupation`: how many units of land are occupied by the building.

Claire knows that the `vector` container of the C++ standard library provides many functionalities she needs. Therefore, she has decided to use the following internal (`private`) representation for the library:

- `enum Purpose { ADMINISTRATION, DEPARTMENT, AUDITORIUM }` — an enumeration to specify the purpose of a building;
- `vector<Building> buildings` — the buildings present in the campus;
- `unsigned int land` — the total units of land owned by the campus.

Claire has already implemented the default constructor of `Campus`, which creates a digital twin of the campus with the existing buildings. She has also implemented the methods `display()`, which shows information on all the buildings in the campus, and `purposeToString()`, which converts a `Purpose` into a `string`.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

- (a) Implement the following method to add a new building to the campus:

```
void Campus::buildNewBuilding(string name, Purpose purpose,
                               unsigned int landOccupation)
```

This exercise continues on the next page...

The method must work as follows:

- (a) if a building with the same `name` already exists, return `false`;
- (b) otherwise:
 - if the land occupied by all the existing buildings plus the new building exceeds 90% of the total land owned by the campus, return `false`;
 - if the land occupied by all the existing buildings plus the new building is less or equal to 90% of the total land owned by the campus, then update the vector `buildings` to register the new building and return `true`.

- (b) Implement the following method to show the buildings for a given purpose:

```
void Campus::printBuildingNames(Purpose purpose)
```

This method displays the name(s) of the building(s) for the specified `purpose`.

The names must be displayed one-per-line, from the one of the smallest building (i.e., the one that occupies the least units of land) to the largest (i.e., the one that occupies the most units of land). In case of buildings occupying the same units of land, their name must be displayed in the same order as the one shown by `display()`.

- (c) Implement the following method to re-purpose a building:

```
bool Campus::repurposeBuilding(string name, Purpose newPurpose)
```

The method must work as follows:

- (a) if a building with the `name` does not exist, return `false`;
- (b) otherwise:
 - if the building with the `name` already has the purpose specified by `newPurpose`, return `false`;
 - otherwise, change the purpose of the building with the `name` to `newPurpose` and return `true`.

- (d) Implement the method:

```
unsigned int Campus::demolishBuildings(vector<string> names)
```

This method removes from the campus all the buildings whose name appears in `names`, and returns how many units of land were freed.

EXERCISE 4. BOUNDED BUFFER

Daisy is writing a program that stores and retrieves `integer` values from a sensor. Her program needs to implement a *bounded* buffer that accumulates values only if they are lower than an upper bound. Therefore, she plans a `BoundedBuffer` class with the following interface:

- `write(v)`: appends value `v` to the buffer only if `v` is lower than the upper bound.
- `read()`: removes the oldest value from the buffer and returns it; if no value is in the buffer, it returns the default value specified in the `BoundedBuffer` constructor (see below).
- `setBound(v)`: sets the upper bound for the buffer to `v`; if the buffer contains one or more values, it removes all the values that are greater than or equal to `v`; the number of values being removed is then returned.
- `occupancy()`: returns the number of elements in the buffer; calling `write(v)` increases the occupancy by 1 if `v` is added to the buffer, calling `read()` decreases the occupancy by 1, while calling `setBound(v)` decreases the occupancy by the number of values being removed.

Her first test program is in file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. These files are available inside the `handin.zip` archive, under `exercise4boundedbuffer` folder.

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write(v)`, `read()`, and `occupancy()`.

Example. Once completed, the class `BoundedBuffer` must work as follows:

- suppose that we create `buf = BoundedBuffer(5,-1)` (i.e. `buf` has an upper bound of 5 and a default value -1);
- suppose that `buf.write(9)` is invoked, followed by `buf.write(1)`. Then, a call to `buf.occupancy()` must return 1, since 9 was not added to the buffer (since it is greater than the upper bound 5), whereas 1 was added to it.
- now, suppose that `buf.read()` is invoked. Then, a call to `buf.occupancy()` must now return 0, since 1 was removed when `buf.read()` was invoked.
- now, suppose that `buf.write(3)` is invoked, followed by `buf.write(8)`, `buf.write(2)` and `buf.write(4)`. Then, a call to `buf.occupancy()` must return 3 since only 8 was not added to the buffer, whereas 2 and 4 were.
- finally, suppose that `buf.setBound(3)` is invoked. Then, a call to `buf.occupancy()` must now return 1, since 3 and 4 were removed from the buffer when `buf.setBound(3)` was invoked (since they are greater than or equal to the new upper bound 3).

This exercise continues on the next page...

Tasks. Help Daisy by completing the following tasks. **IMPORTANT:** for these tasks you are required to submit *both* files `ex04-library.h` and `ex04-library.cpp` enclosed in a ZIP archive.

NOTE: you are free to define the `private` members of `BoundedBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write(v)`, `read()`, `occupancy()` and `setBound(v)`.

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `BoundedBuffer` that extends `Buffer`. This task is completed (and passes the tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a constructor for `BoundedBuffer` that takes 2 parameters:
 - (i) an `integer` representing the initial upper bound; and
 - (ii) an `integer` representing a default value (it is used in point (c) below);
 2. in `BoundedBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` method to the class interface:
 - `unsigned int setBound(int v)`
 3. finally, write a placeholder implementation of all the `BoundedBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).
- (b) This is a follow-up to point (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void BoundedBuffer::write(int v)
unsigned int BoundedBuffer::occupancy()
```

Their intended behaviour is that each time `write()` is invoked, it adds the value `v` to the buffer only if it is lower than the upper bound (such upper bound is initially specified in the constructor — see point (a)1(i) above); if `v` is added to the buffer, then the value returned by `occupancy()` increases by 1.

This exercise continues on the next page...

- (c) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int BoundedBuffer::read()
```

When `read()` is invoked, it removes the oldest value previously added by `write()`, and returns it; correspondingly, the value returned by `occupancy()` decreases by 1.

Special case: if the buffer is empty, then `read()` must return the default value specified in the constructor (see point (a)1(ii) above).

- (d) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
unsigned int BoundedBuffer::setBound(int v)
```

When `setBound(v)` is invoked, it sets the upper bound for the buffer to `v`, and then removes from the buffer all the values greater than `v` (if any); it then returns the number of values being removed from the buffer; correspondingly, the value returned by `occupancy()` decreases by the same value as the value returned by `setBound(v)`.